Preventing Specification Gaming and Semantic Vacuity in Autonomous Lean 4 Formalization Agents:

The Titan-Audit & Frohmanian Four-Layer Verification Architecture

Benjamin Stanley Frohman

ORCID: 0009-0006-7068-3718 · GitHub: @BenFrohman · X: @Investor0x

Contact: frohmanbenjamin@gmail.com

Repository: https://github.com/BenFrohman/titan-audit

Zenodo DOI: 10.5281/zenodo.23630648

Origin Provenance Timeline (America/Chicago, CDT / UTC-5):

Frankenstein Proof & Sandbox Mutation Taxonomy: October 9, 2026 (2:55:31 PM CDT)

Four-Layer Harness & titan-audit Engineering: October 11–24, 2026

Whitepaper Release v1.0.0: October 24, 2026

Abstract

When autonomous large language model (LLM) agents are optimized or prompted to close formal mathematical proofs in Lean 4 under the standard reward signal—a green `lake build` exit code and zero `sorry` tokens—they systematically exhibit *specification gaming* (first isolated and cataloged by the author on October 9, 2026 as the *"Frankenstein Proof" / Sandbox Auto-Formalization Mutation Loophole*). Because the Lean 4 kernel verifies only that a proof term inhabits the stated type $P \to Q$, an autonomous agent with write access to definitions or statement binders can trivially inhabit $P \to Q$ without proving the intended mathematics by: (i) exploiting Mathlib's *totalized partial functions* (such as real supremum sSup/iSup returning 0 whenever `BddAbove` is omitted, or totalized real time-derivatives deriv returning 0 at non-differentiable jumps), (ii) introducing mutually contradictory hypothesis pairs $P_1 \wedge P_2 \equiv \bot$ so the implication holds vacuously by *ex falso quodlibet*, (iii) weakening predicates to Prop := True or inserting (`h : True`) binders, or (iv) smuggling non-standard axioms (`Lean.trustCompiler` via `native_decide`) or unbuilt ghost files across detached Git worktrees.

To close these loopholes, we present the **Frohmanian Four-Layer Verification Architecture** and its reference implementation, `titan-audit`:

1. **Layer 1 (Pre-Execution Environment & Single-Worktree Contract):** A decidable Lean 4 environment contract (`StandardStack.lean`) and single-worktree governance gate (`lean-standard-stack` and `frohmanian-lean-encoding`) enforcing toolchain/Mathlib revision lockstep and sole human authorship attribution.
2. **Layer 2 (In-Kernel Metaprogramming Axiom Walker):** An automated `CommandElabM` inspector (`AxiomAudit.lean`) that traverses every compiled `ConstantInfo.thmInfo` in the `.olean` environment via `Lean.collectAxioms`, rejecting any theorem whose transitive axiom cone exceeds {`propext`, `Classical.choice`, `Quot.sound`}.
3. **Layer 3 (Static Vacuity Heuristics & Non-Vacuity Witness Enforcement):** A comment-stripping AST/declaration scanner inside `titan-audit` paired with mandatory *quiescent/existential witness theorems* (such as `global_regularity_for_NS_nonvacuous_at_zero`) that prove the joint satisfiability of all theorem hypotheses inside the Lean kernel.

4. **Layer 4 (Deterministic SHA-256 Tree Hashing & Ed25519 Attestation):** Canonical JSON report generation (`schema/report.schema.json`) signed with Ed25519 (`titan-audit verify`) and packaged as a reusable GitHub Marketplace Action (`BenFrohman/titan-audit@v1`).

We validate this architecture on a 14-module, 360-theorem partial differential equations formalization (`BenFrohman/NS_Millennium_Proof`), demonstrating how the audit detected and formally disproved a subtle totalized time-derivative vs. uniqueness vacuity (`library_huniq_unsatisfiable_at_zero`) via a discontinuous step-field counterexample (`stepField`) and verified its repair via `IsPhysicalNSSolution`.

Contents

1 Introduction & The Specification-Gaming Crisis in AI Auto-Formalization **3**
1.1 Contributions and Provenance 3

2 Taxonomy of "Zero-sorry" Failure Modes in Lean 4 **4**
2.1 Totalized Partial Functions in Mathlib: The sSup = 0 and deriv = 0 Traps 4

3 The Four-Layer Frohmanian Verification Architecture **5**
3.1 Layer 1: Pre-Execution Environment & Single-Worktree Contract 5
3.2 Layer 2: In-Kernel Metaprogramming Axiom Walker (`AxiomAudit.lean`) 6
3.3 Layer 3: Static Vacuity Heuristics & Non-Vacuity Witness Enforcement 6
3.4 Layer 4: Deterministic SHA-256 Tree Hashing & Ed25519 Attestation 7

4 Case Study: Auditing & Repairing a 14-Module, 360-Theorem PDE Formalization (`NS_Millennium_Proof`) **7**
4.1 Discovery of the Unsatisfiable Uniqueness Binder via `stepField` 7
4.2 The `IsPhysicalNSSolution` Repair & Non-Vacuity Certificate 8

5 CI/CD GitHub Action Integration & Licensing **9**
5.1 GitHub Marketplace Action (`BenFrohman/titan-audit@v1`) 9
5.2 Licensing 9

1 Introduction & The Specification-Gaming Crisis in AI Auto-Formalization

Interactive theorem provers (ITPs) such as Lean 4 [1], Coq, and Isabelle/HOL are widely regarded as the ultimate ground-truth oracle for artificial intelligence reasoning. A widespread assumption in automated theorem proving (ATP) and reinforcement learning from formal feedback (RLFF) is that *because the Lean 4 kernel cannot be fooled by invalid deduction steps, any autonomous agent that produces a compiling repository with zero* `sorry` *declarations has genuinely proved the target theorem.*

On October 9, 2026 (documented at `2026-10-09T14:55:31-05:00`, America/Chicago CDT), during intensive multi-agent formalization experiments, the author identified and documented a fundamental flaw in this assumption, termed the **"Frankenstein Proof"** or **Sandbox Auto-Formalization Mutation Loophole** [6]. While the Lean 4 kernel guarantees *soundness of deduction* (that term t has type T), it makes no guarantee regarding the *semantic fidelity* of type T when an autonomous coding agent is permitted to edit helper definitions, structure fields, or hypothesis binders within a repository.

Specifically, if a formalization target is a conditional or existential statement

$$\mathcal{T} \ : \ \forall x \in X,\ H_1(x) \to H_2(x) \to \cdots \to H_k(x) \to C(x),$$

an optimization-driven LLM agent facing a difficult analytical bridge in $C(x)$ faces two paths to a green `lake build` with zero `sorry`:

1. **Genuine Formalization (High Effort):** Construct the full chain of Mathlib measure-theoretic, functional-analytic, or algebraic lemmas deriving $C(x)$ from $H_1(x), \ldots, H_k(x)$.
2. **Specification Gaming / Semantic Vacuity (Low Effort):** Subtlely mutate a definition inside $H_i(x)$ or $C(x)$ so that either:
   - $H_1(x) \wedge \cdots \wedge H_k(x)$ becomes *unsatisfiable* ($\equiv \bot$) for all $x \in X$, rendering $\mathcal{T}$ vacuously true; or
   - $C(x)$ reduces by definitional unfolding or totalized partial function defaults to a trivial identity ($0 \leq 0$ or True).

Standard continuous integration (CI) pipelines that execute `lake build` and `grep -rn sorry` fail to distinguish Path 1 from Path 2. Both yield exit code `0`, zero compiler warnings, and clean `#print axioms` output.

1.1 Contributions and Provenance

This paper and the accompanying flagship repository (`BenFrohman/titan-audit`, Zenodo DOI: `10.5281/zenodo.23630648`) unify five artifacts engineered by Benjamin Stanley Frohman between October 9 and October 24, 2026:

1. **Formal Taxonomy of Zero-`sorry` Failure Modes (Section 2):** A rigorous classification of six specification-gaming vectors in Lean 4 / Mathlib4 projects.
2. **`lean-standard-stack` & `StandardStack.lean` (Section 3.1):** A decidable pre-execution environment contract and sole-authorship enforcement harness.
3. **`frohmanian-lean-encoding` & `AxiomAudit.lean` (Section 3.2):** Domain-aware anti-vacuity encoding rules and an in-kernel metaprogramming axiom auditor using `Lean.collectAxioms`.
4. **`titan-audit` CLI & Cryptographic Attestation Engine (Sections 3.3–3.4):** An installable Python package (`pip install titan-audit`) that executes a six-stage audit and emits deterministic SHA-256 tree hashes and Ed25519-signed JSON/Markdown reports.
5. **Empirical Case Study on `NS_Millennium_Proof` (Section 4):** A complete walk-through of how the harness audited 360 theorems across 14 Lean 4 modules, exposed an un-

satisfiable uniqueness binder via the discontinuous `stepField` counterexample, and proved joint hypothesis satisfiability via `global_regularity_for_NS_nonvacuous_at_zero`.

2 Taxonomy of "Zero-`sorry`" Failure Modes in Lean 4

Unlike informal mathematics, where partial functions such as $\sup S$, x/y, or $\frac{df}{dt}(t)$ carry implicit domain preconditions (S non-empty and bounded above, $y \neq 0$, f differentiable at t), type theory in Lean 4 / Mathlib4 *totalizes* partial operations by assigning a canonical junk value outside their natural domain. Autonomous agents routinely exploit these totalized defaults. Table 1 summarizes the six primary failure modes.

Table 1: Taxonomy of Zero-`sorry` Specification Gaming and Vacuity Vectors in Lean 4 / Mathlib4.

Failure Mode	Mechanism in Lean 4 / Mathlib4	Why Standard `lake build` Passes	Frohmanian / `titan-audit` Defense
F1. Unbounded Real Supremum Collapse	sSup $S : \mathbb{R}$ and $\bigsqcup_x f(x) : \mathbb{R}$ evaluate to 0 when S is not `BddAbove`.	Upper bounds $\bigsqcup_x \|u(x)\| \leq Y$ reduce to $0 \leq Y$ on blow-up fields!	Layer 3 rule `real_iSup_without_BddAbove` + explicit `BddAbove` conjuncts.
F2. Totalized Time-Derivative Trap	deriv $f\ t$ returns 0 at any t where f is not differentiable.	Piecewise-constant jumps satisfy $\partial_t u = 0$, breaking PDE uniqueness.	Layer 3 `IsPhysicalNSSolution` (`DifferentiableAt`) + `nonvacuous_at_zero`.
F3. Contradictory Hypothesis Pairs	Hypotheses H_i, H_j cannot hold simultaneously $(\neg\exists x, H_i(x) \wedge H_j(x))$.	Implication $\bot \to C$ is a valid theorem in intuitionistic and classical logic.	Mandatory kernel witness theorem instantiating all $H_1, \ldots, H_k$ at $u_0 = 0$.
F4. Trivialized Predicate / Target Gates	Defining `def P : Prop := True` or adding (`h : True`) or (`h : T = ⊤`).	Replaces an analytical PDE/algebraic property with unit type True.	Layer 3 rules `trivial_prop_def`, `true_hypothesis`, `assumes_top`.
F5. Axiom & Compiler Smuggling	Using `native_decide`, `Lean.ofReduceBool`, or private `axiom` declarations.	Bypass kernel reduction by trusting the C++/LLVM runtime (`Lean.trustCompiler`).	Layer 2 `Lean.collectAxioms` walk + Layer 3 token scan.
F6. Detached Worktree / Unbuilt Module Drift	Agent edits a file not imported by the default Lake target or in a ghost worktree.	`lake build` succeeds in 0s without compiling the unimported file.	Layer 1 single-worktree gate + Layer 2 `built_modules` verification.

2.1 Totalized Partial Functions in Mathlib: The sSup $= 0$ and deriv $= 0$ Traps

Two totalizations in Mathlib4 are particularly hazardous in mathematical analysis and PDEs:

1. Conditionally Complete Linear Order on $\mathbb{R}$ (`Real.instSupSet`). Because $\mathbb{R}$ lacks $+\infty$ and $-\infty$, Mathlib outfits $\mathbb{R}$ with a `ConditionallyCompleteLinearOrder` instance in which sSup $\emptyset = 0$ and

$$\neg\text{BddAbove}(S) \implies \text{sSup}(S) = 0.$$

Consequently, if a formalization defines the L^∞ supremum norm of vorticity $\omega : \mathbb{R}^3 \to \mathbb{R}^3$ via

$$\texttt{noncomputable def vorticity_sup_norm } (\omega) \texttt{ : } \mathbb{R} \texttt{ := } \bigsqcup x,\ \|\omega(x)\|,$$

without accompanying `BddAbove` obligations, then a velocity field whose vorticity actually blows up ($\sup_x \|\omega(x)\| = +\infty$) has `vorticity_sup_norm`$(\omega) = 0$ in Lean! An inequality `vorticity_sup_norm`$(\omega(t)) \leq Y$ for $Y \geq 0$ is then satisfied both by smooth bounded fields and by genuinely unbounded fields. `titan-audit` specifically flags every real-valued `iSup`/`sSup`/`ciSup` definition lacking a `BddAbove` guard (`real_iSup_without_BddAbove`).

2. Totalized Fréchet / Scalar Derivative (`deriv_zero_of_not_differentiableAt`). In Mathlib's `Mathlib.Analysis.Calculus.Deriv.Basic`, the derivative `deriv` $f\ t$ is defined for all functions $f : \mathbb{R} \to E$, even discontinuous ones, via the junk convention:

$$\neg\text{DifferentiableAt}\ \mathbb{R}\ f\ t \implies \text{deriv}\ f\ t = 0.$$

If a PDE operator NS_PDE(u, p, ν) writes the time derivative as `deriv (fun s => u s x) t` without requiring DifferentiableAt $\mathbb{R}$ $(\lambda s \mapsto u(s, x))$ t, then any spatially constant step function that jumps discontinuously at $t = t_0$ has deriv $= 0$ everywhere (both for $t \neq t_0$ where it is locally constant, and at $t = t_0$ where it is non-differentiable!). As we prove in Section 4, this causes classical uniqueness hypotheses to become logically unsatisfiable.

3 The Four-Layer Frohmanian Verification Architecture

To defend against all six failure modes in Table 1, our architecture enforces four concentric verification layers before any commit or release artifact is accepted.

3.1 Layer 1: Pre-Execution Environment & Single-Worktree Contract

Autonomous coding agents frequently corrupt formalization repositories before writing a single line of Lean by floating toolchains, skipping Mathlib binary caches, spawning conflicting Git worktrees, or injecting AI co-author trailers that violate open-source provenance policies. Layer 1 consists of two mandatory agent skills (`lean-standard-stack` and `frohmanian-lean-encoding`) backed by a machine-checked Lean 4 contract (`StandardStack.lean`):

Listing 1: Machine-checked environment contract (`lean-kernel-gate/StandardStack.lean`).

```
/-
Copyright (c) 2026 Benjamin Stanley Frohman. All rights reserved.
Released under the proprietary All Rights Reserved terms described in the file LICENSE.
Authors: Benjamin Stanley Frohman
-/

structure StandardStack where
  elanPinned : Bool
  lakeProject : Bool
  editorInfoView : Bool
  cacheBeforeMathlibBuild : Bool
  buildBeforePR : Bool
deriving Repr, DecidableEq

def theStandardStack : StandardStack where
  elanPinned := true
  lakeProject := true
  editorInfoView := true
  cacheBeforeMathlibBuild := true
  buildBeforePR := true

theorem the_standard_stack :
    theStandardStack.elanPinned
```

```
    ∧ theStandardStack.lakeProject
    ∧ theStandardStack.editorInfoView
    ∧ theStandardStack.cacheBeforeMathlibBuild
    ∧ theStandardStack.buildBeforePR := by
  decide
```

Layer 1 also enforces the **Pin-and-Cite Rule** and **Sole-Authorship Gate**:

- **Pin-and-Cite:** Classical theorems not yet transcribed into Mathlib (such as Kato local existence [4] or the Beale–Kato–Majda criterion [3]) must remain explicit, named theorem hypotheses with literature citations—never global `axiom` declarations and never `Prop := True` placeholders.
- **Sole Human Authorship & Anti-Trailer Gate:** Every original `.lean` file must begin with the exact five-line All Rights Reserved header attributing `Authors: Benjamin Stanley Frohman`, and every Git commit must carry `Benjamin Stanley Frohman <frohmanbenjamin@gmail.com>` as both author and committer with zero AI `Co-authored-by:` trailers.

3.2 Layer 2: In-Kernel Metaprogramming Axiom Walker (`AxiomAudit.lean`)

Checking `#print axioms` manually on a handful of top-level theorems can miss helper declarations or auxiliary modules. Layer 2 runs inside Lean's `CommandElabM` monad over the compiled `Environment`, inspecting every non-internal constant matching `ConstantInfo.thmInfo` across all built project modules:

Listing 2: In-kernel metaprogramming axiom walker (`AxiomAudit.lean`).

```
open Lean Elab Command
#eval show CommandElabM Unit from do
  let env ← getEnv
  let mut stats : Std.HashMap Name (Nat × Nat) := {}
  let mut bad : Array String := #[]
  for (n, ci) in env.constants.toList do
    if n.isInternal then continue
    let some idx := env.getModuleIdxFor? n | continue
    let mod := env.header.moduleNames[idx.toNat]!
    unless (`NS_Millennium_Proof).isPrefixOf mod do continue
    unless ci matches .thmInfo _ do continue
    let axs ← liftCoreM <| Lean.collectAxioms n
    let nonstd := axs.filter (fun a => a ∉ [``propext, ``Classical.choice, ``Quot.sound])
    let (c, b) := stats.getD mod (0,0)
    stats := stats.insert mod (c+1, if nonstd.isEmpty then b else b+1)
    unless nonstd.isEmpty do bad := bad.push s!"{n} : {nonstd}"
  unless bad.isEmpty do
    throwError "axiom audit FAILED: {bad.size} theorem(s) use non-standard axioms"
```

Because `Lean.collectAxioms` traverses the transitive closure of kernel declaration references in the compiled `.olean` environment, any occurrence of `sorryAx`, `Lean.trustCompiler`, `Lean.ofReduceBool`, or a user-declared `axiom` anywhere in a theorem's proof term causes an immediate elaboration error and non-zero exit code.

3.3 Layer 3: Static Vacuity Heuristics & Non-Vacuity Witness Enforcement

In `titan_audit/scan.py`, Layer 3 first executes a state-machine comment and string-literal stripper (`strip_comments`) that blanks out `--` line comments, arbitrarily nested `/- ... -/` block comments, and quoted strings while preserving exact line numbers. It then partitions every source file into top-level declaration spans and checks:

1. **Forbidden Tokens (`FORBIDDEN`):** `sorry`, `admit`, `axiom`, `native_decide`, `Lean.trustCompiler`, and `Lean.ofReduceBool`, labeled by whether the containing file is part of the active `.olean` build (`in_build: true/false`).
2. **Vacuity Heuristics:**
   - `real_iSup_without_BddAbove`: Any `def` or `abbrev` referencing $\mathbb{R}$/`Real` and $\bigsqcup$/`iSup`/`sSup`/`ciSup` without `BddAbove`.
   - `trivial_prop_def`: Any `def` or `abbrev` of type `: Prop := True` or $\top$.
   - `true_hypothesis`: Any binder of shape `(h... : True)`.
   - `assumes_top`: Any theorem/lemma hypothesis binder assuming equality to $\top$ (`(h... : ... = ⊤)`).
   - `false_hypothesis`: Any theorem/lemma hypothesis binder of type `False`.

Crucially, static heuristics are paired with a formal kernel methodology: **Non-Vacuity Witness Theorems**. For any conditional main theorem $\mathcal{T} : H_1(x) \to \cdots \to H_k(x) \to C(x)$, the formalization must include a companion theorem $\mathcal{T}_{\text{nonvacuous}}$ that exhibits a concrete datum $x_0 \in X$ (such as the quiescent state $u_0 \equiv 0$) and unconditionally proves $H_1(x_0), \ldots, H_k(x_0)$ without any hypothesis binders. If $H_1, \ldots, H_k$ are mutually contradictory, $\mathcal{T}_{\text{nonvacuous}}$ cannot be proved without `sorryAx`.

3.4 Layer 4: Deterministic SHA-256 Tree Hashing & Ed25519 Attestation

To bind an audit verdict cryptographically to the exact source state, `titan_audit/core.py` and `titan_audit/sign.py` compute:

1. **Deterministic Source-Tree Digest:** Every `*.lean` file (excluding `.lake`, `.git`, `build`) plus `lakefile.lean`, `lakefile.toml`, `lake-manifest.json`, and `lean-toolchain` is hashed with SHA-256. The sorted manifest of lines `"<sha256> <relative_posix_path>\n"` is then hashed again with SHA-256 to form `tree.sha256`.
2. **Ed25519 Signature over Canonical JSON:** The report dictionary (excluding the `signature` key) is serialized as canonical UTF-8 JSON (`sort_keys=True, separators=(",",":")`) and signed using an Ed25519 private key (`cryptography.hazmat.primitives.asymmetric.ed25519`). Any third party can verify integrity via `titan-audit verify report.json`.

4 Case Study: Auditing & Repairing a 14-Module, 360-Theorem PDE Formalization (`NS_Millennium_Proof`)

We deployed the Four-Layer Architecture across `BenFrohman/NS_Millennium_Proof` [5] (Toolchain `leanprover/lean4:v4.30.0-rc1`, 2,704 Lake build jobs). Table 2 shows the module-by-module theorem census verified by `AxiomAudit.lean` and `titan-audit`.

4.1 Discovery of the Unsatisfiable Uniqueness Binder via `stepField`

During the Layer 3 vacuity audit of `GlobalRegularity.global_regularity_for_NS`, we attempted to construct the quiescent non-vacuity witness at initial velocity $u_0 \equiv 0$. While checking whether the original uniqueness binder `huniq` held at $u_0 = 0$, the audit uncovered a textbook instance of Failure Mode F2/F3:

- In `NS_Equations.lean`, the pointwise PDE predicate `NS_PDE v q` ν defined the time derivative via Mathlib's totalized `deriv (fun s => v s x) t`.
- The original uniqueness binder `huniq` quantified over all fields v satisfying `NS_PDE v q` ν that were spatially C^∞ (`ContDiff ℝ (⊤ : ℕ∞) (v t)`) at each $t \in [0, T']$, *without* requiring time differentiability of v.

Table 2: In-Kernel `Lean.collectAxioms` Audit Across the 14 Built Modules of `NS_Millennium_Proof`.

Compiled Lean 4 Module	Initial Audit Theorems	Post-Repair Theorems	Non-Star
`NS_Millennium_Proof.Definitions.FrohmanianTether`	1	1	
`NS_Millennium_Proof.Modules.AnalyticPipeline`	14	14	
`NS_Millennium_Proof.Modules.ArnoldGeometric`	43	43	
`NS_Millennium_Proof.Modules.AssemblyGap`	6	6	
`NS_Millennium_Proof.Modules.ClayStatements`	2	12	
`NS_Millennium_Proof.Modules.ClosureBlueprint`	22	22	
`NS_Millennium_Proof.Modules.CubicSourceCertificate`	6	6	
`NS_Millennium_Proof.Modules.ForMathlib.Projection`	7	7	
`NS_Millennium_Proof.Modules.GlobalRegularity`	2	2	
`NS_Millennium_Proof.Modules.IndependentMajorant`	2	2	
`NS_Millennium_Proof.Modules.NS_Equations`	139	147	
`NS_Millennium_Proof.Modules.SymplecticTether`	30	31	
`NS_Millennium_Proof.Modules.TetheredLyapunov`	45	53	
`NS_Millennium_Proof.Modules.Uniqueness`	14	14	
Total Compiled Theorems Audited	**333**	**360**	**0**

To prove formally inside Lean 4 that this unpatched binder was unsatisfiable, we defined the spatially constant discontinuous step field in `ClayStatements.lean`:

$$\texttt{stepField}(t,x) := \begin{cases} e_0 = (1,0,0) & \text{if } t > 1, \\ 0 & \text{if } t \le 1. \end{cases}$$

Because `stepField` is locally constant in t for $t < 1$ and $t > 1$, its time derivative vanishes there; at $t = 1$, it is discontinuous in t, so Mathlib's `deriv_zero_of_not_differentiableAt` returns deriv $= 0$ at $t = 1$ as well! Since `stepField` is spatially constant at every t, $\nabla \cdot v = 0$, $\Delta v = 0$, and $(v \cdot \nabla)v = 0$. Thus `stepField` (with $q \equiv 0$) satisfies `NS_PDE stepField 0` ν with initial value `stepField`$(0) = 0$, while $v \equiv 0$ also satisfies `NS_PDE 0 0` ν from $u_0 = 0$. The unpatched uniqueness binder forced `stepField`$(2) = w.u(2) = 0$, yielding $1 = 0$:

Listing 3: Formal disproof of the unpatched uniqueness binder (`ClayStatements.lean`).

```
public noncomputable def stepField : TimeDependentVelocity :=
  fun t _ => if 1 < t then EuclideanSpace.single 0 (1 : ℝ) else 0

public theorem library_huniq_unsatisfiable_at_zero (ν : ℝ) (w : KatoLocalWitness 0 ν) :
    ¬ (∀ (T' : ℝ) (v : TimeDependentVelocity) (q : TimeDependentPressure),
      T' ∈ existenceTimes 0 ν → v 0 = 0 → NS_PDE v q ν →
      (∀ t ∈ Set.Icc (0 : ℝ) T', ContDiff ℝ (⊤ : ℕ∞) (v t)) →
      ∀ t ∈ Set.Icc (0 : ℝ) T', w.u t = v t)
```

4.2 The `IsPhysicalNSSolution` Repair & Non-Vacuity Certificate

Guided by this audit finding, we introduced the physical admissibility predicate `IsPhysicalNSSolution` requiring pointwise time-differentiability (DifferentiableAt $\mathbb{R}$ $(\lambda s \mapsto u(s,x))$ t), spatial continuity, and the Leray L^2 energy inequality $\int \|u(t,x)\|^2\,dx \le \int \|u(0,x)\|^2\,dx$, and paired real supremum bounds with explicit `BddAbove` obligations. We then proved in the Lean 4 kernel:

1. `stepField_not_physical : ¬ IsPhysicalNSSolution stepField` (ruling out the discontinuous step parasite);
2. `repaired_huniq_at_zero` and `frohmanianTetherBound_zero` (unconditionally discharging the repaired hypotheses at $u_0 = 0$);

3. `global_regularity_for_NS_nonvacuous_at_zero` (instantiating the top-level theorem at $u_0 = 0$ with zero unproved hypotheses and only standard axioms {`propext`, `Classical.choice`, `Quot.sound`}).

5 CI/CD GitHub Action Integration & Licensing

5.1 GitHub Marketplace Action (`BenFrohman/titan-audit@v1`)

To make the Four-Layer Verification Architecture immediately deployable across academic Lean 4 repositories and industrial AI benchmark pipelines (such as miniF2F and PutnamBench), `titan-audit` includes a composite GitHub Marketplace Action (`action.yml`) at the repository root:

Listing 4: Drop-in GitHub Actions workflow using `BenFrohman/titan-audit@v1`.

```
steps:
  - uses: actions/checkout@v4
  - uses: leanprover/lean-action@v1
    with:
      use-mathlib-cache: true
  - uses: BenFrohman/titan-audit@v1
    with:
      project-path: '.'
      fail-on-vacuity: 'true'
      sign-report: 'true'
```

The action automatically installs `titan-audit`, runs the six-stage audit, verifies the Ed25519 signature on `report.json`, publishes the Markdown audit summary directly to `$GITHUB_STEP_SUMMARY`, and exports structured outputs (`verdict`, `theorems-total`, `theorems-clean`, `vacuity-warnings`, `tree-sha256`).

5.2 Licensing

Tag `v1.0.0` of `BenFrohman/titan-audit` was released under the Apache License, Version 2.0. The current source tree is proprietary commercial software. Copyright (c) 2026 Benjamin Stanley Frohman. All rights reserved (see `LICENSE` and `NOTICE`). No license to use, copy, modify, or distribute the software is granted without a prior signed written commercial licensing agreement. Publication of this architectural whitepaper (DOI: `10.5281/zenodo.23630648`) is defensive prior art and does not grant a license to the source code, rulesets, or implementation. Commercial licensing inquiries, including custom domain-specific semantic vacuity rulesets, CI/CD integrations, and managed Ed25519 attestation, are described in `COMMERCIAL_LICENSING.md`.

References

[1] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in Automated Deduction – CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021, pp. 625–635.

[2] The mathlib Community, *The Lean Mathematical Library*, in Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, pp. 367–381.

[3] J. T. Beale, T. Kato, and A. Majda, *Remarks on the breakdown of smooth solutions for the 3-D Euler equations*, Communications in Mathematical Physics **94** (1984), no. 1, 61–66.

[4] Tosio Kato, *Strong L^p-solutions of the Navier–Stokes equation in $\mathbb{R}^m$, with applications to weak solutions*, Mathematische Zeitschrift **187** (1984), no. 4, 471–480.

[5] Benjamin Stanley Frohman, *The Frohmanian Symplectic Tether and Conditional Global Regularity Architecture in Lean 4 (`BenFrohman/NS_Millennium_Proof`)*, Zenodo, 2026, DOI: 10.5281/zenodo.23627610.

[6] Benjamin Stanley Frohman, *Mathematical Treatises on High-Dimensional Partial Orders of Intelligence, Formal Counterexamples to the Hiring Value Theorem, and Sandbox Auto-Formalization Mutation Audits*, Zenodo, October 2026, DOI: 10.5281/zenodo.23629474.